

Ship with confidence. *Keep the service reliable.*

Turn a working AI application into a service,
with measured releases and a tested way to recover.

🕒 15-20 min core reading · 20-30 min practice · 12 short sections

Your movie assistant works on your laptop. Then several people use it together, the model provider slows down, and a prompt update starts recommending unsuitable films. Production is the work of keeping the whole service useful while traffic, dependencies, and application versions change.

WHAT YOU WILL UNDERSTAND

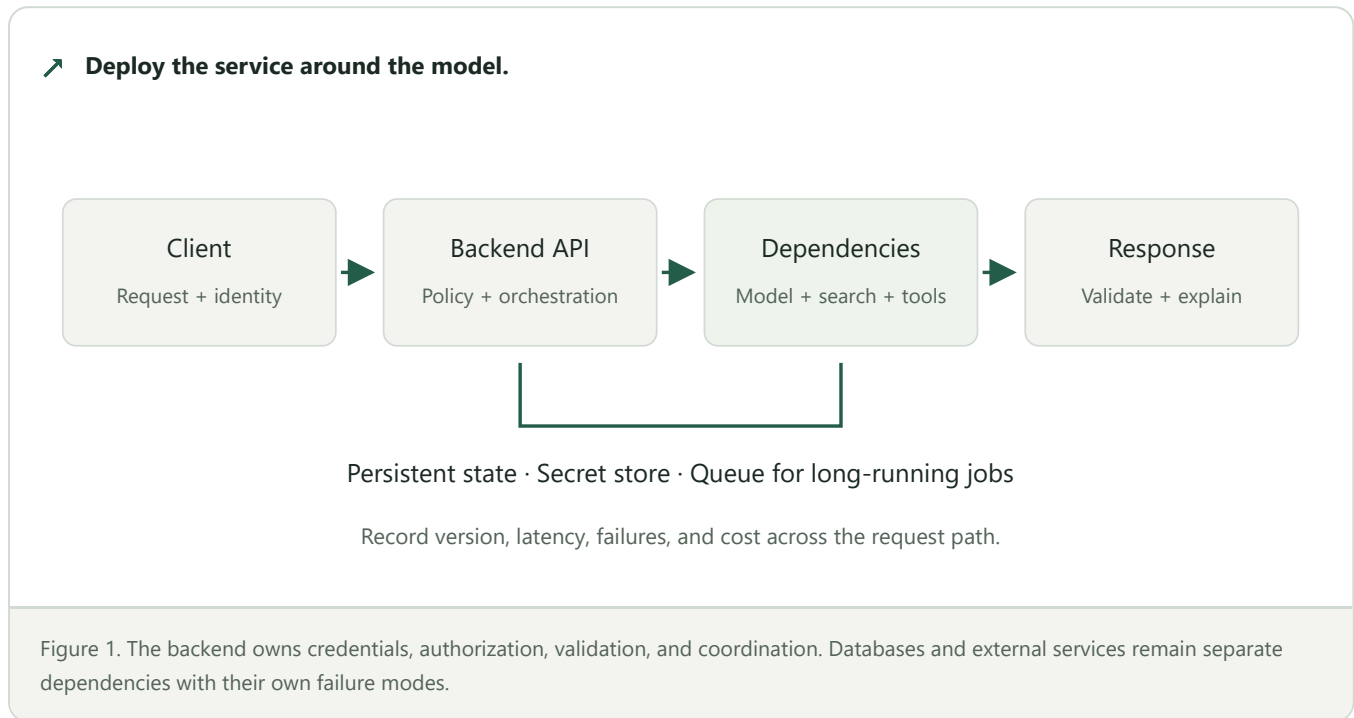
The main question: how do you release an AI application, know whether it is working, and recover when it is not?

The practical path: package a version, test its dependencies, release gradually, measure user outcomes, and operate with explicit limits and recovery steps.

Reading path: twelve short sections, three diagrams, a release simulator, and a Python workbook. Optional notes explain the operational tradeoffs.

1. Deploy a service, not just a model call

Deployment makes an application version available in an environment. **Operations** keeps that service functioning over time. A hosted model API operates the model for you, but you still own your application, account permissions, retrieval data, user experience, and failure handling. A self-hosted model also makes you responsible for model serving capacity and infrastructure.



For the movie assistant, a browser sends a request to a backend API. The backend checks identity and inputs, retrieves permitted movie facts, calls the model, validates the result, and returns a response. Persistent data such as watchlists belongs in an appropriate store. Long-running catalog ingestion can use a queue and worker so a browser request need not wait for the entire job.

A **container** packages an application and its runtime dependencies into a deployable unit; it does not include every external service or guarantee isolation by itself. A managed application platform, virtual machine, or container service can run that unit. Serverless execution can simplify operations for suitable workloads, but time limits, cold starts, and connection handling still affect behavior. Choose the smallest setup that meets the application's requirements.

Keep secrets on the backend, supplied through a secret manager or protected runtime configuration. Use separate development, staging, and production environments, credentials, and data access. OpenAI's [production guidance](#) discusses secret handling, scaling, and ongoing model operations; this chapter's design is provider-independent.

2. Version everything that can change behavior

A release is more than a Python file. The same code with a new prompt, model configuration, or index can produce different answers. Keep a **release manifest**: a record connecting a deployable artifact to the versions and settings used to evaluate it.

Record	Movie-assistant example
Application and dependencies	Code revision, dependency lockfile, built image digest.
Model and generation settings	Provider/model identifier, sampling settings, output limit.
Prompt and policy	Recommendation instructions, tool schemas, permission rules.
Retrieval and data	Catalog snapshot, chunking/embedding configuration, index version.
Runtime configuration	Timeouts, retry limits, feature flags, routing rules.
Validation evidence	Test results, evaluation dataset version, known limitations.

Use a model snapshot identifier where the service supports one; record unavoidable moving dependencies and retest changes. A pinned identifier improves change tracking but does not make stochastic outputs identical. Validate required configuration at startup, and keep secret values out of manifests and logs.

Continuous integration (CI) builds and checks a change. **Continuous delivery** keeps a checked artifact ready for release; **continuous deployment** automatically releases changes that pass the configured process. Promote the tested artifact across environments rather than rebuilding an unknown variant at the final step. Keep infrastructure and configuration changes reviewable too.

▼ Go deeper: rollback needs compatible state

Restoring yesterday's application is unsafe if today's database migration removed a field it still requires. Use compatible staged migrations: add new structure, support the transition, move usage, and remove obsolete structure only when old versions are no longer needed. Version indexes and retain a tested switching path where practical. A rollback changes future behavior; it does not undo messages already sent, watchlist edits already made, or corrupted data. Backups need tested restoration procedures, not just a successful "backup created" log line.

3. Release gradually and keep a recovery path

Start with unit and integration checks, chapter 34's quality evaluation, and chapter 35's security checks. Staging should exercise the real configuration shape and important integrations using controlled data. Run a **smoke test** for a small set of essential actions after deployment; test load separately to find capacity and latency limits.

➤ **Expose a change gradually. Keep a way back.**

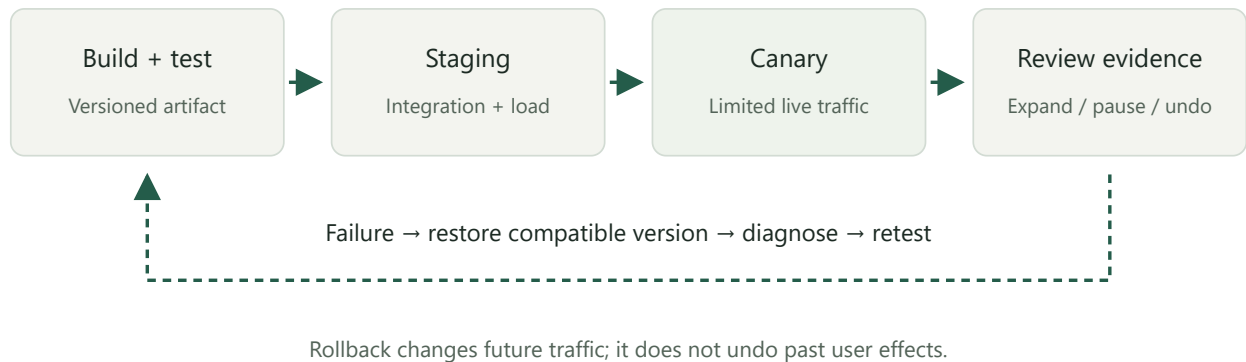


Figure 2. A tested artifact reaches staging, then limited production traffic. Observations determine whether to expand, pause, or return future traffic to a compatible earlier release.

Release pattern	How it works	Important limitation
Rolling update	Replace instances progressively.	Old and new versions coexist; interfaces must remain compatible.
Blue-green	Keep two environments and switch traffic.	Extra capacity and shared-state compatibility still matter.
Canary	Send a limited share to the candidate and inspect outcomes.	Small or biased samples can miss serious defects.
Feature flag	Enable or disable a capability through configuration.	Flag combinations need testing and eventual cleanup.

Set observation periods and stop criteria before rollout. Compare candidate and baseline quality, errors, latency, cost, and important user slices. Google’s [canary-release guidance](#) explains how limited exposure can reduce the impact of a faulty release. A passing canary reduces uncertainty; it cannot establish that every user scenario works.

▼ **Go deeper: representative traffic and observation time**

Assign traffic consistently at the appropriate unit, such as user or session, when switching versions mid-conversation would be misleading. Check language, region, device, and difficult-task coverage. Some failures emerge only after caches expire or background jobs run, so five quick successes cannot justify universal promotion. A shadow run can compare outputs without showing them to users, but must isolate writes and protect user data. Our lab uses fixed request slots and immediate quality labels for reproducibility; real rollout evidence is often delayed, incomplete, and noisy.

4. Observe the request from arrival to outcome

Metrics summarize measurements, **logs** record events, and **traces** connect the steps of a request. A trace contains **spans** for operations such as catalog search and a model call. A correlation or trace ID helps connect the relevant records. These signals are described in [OpenTelemetry's documentation](#).

For “Find a calm movie under 100 minutes,” record the release identifier, retrieval duration, model duration, retries, token usage, validation result, and final outcome. Do not store full private conversations merely because tracing supports them. Minimize logged content and apply access and retention rules.

Signal	What it reveals
Request volume, errors, queue length	Demand, failures, and work accumulating.
Time to first response and total completion time	How quickly the interaction starts and the task finishes.
Task quality and unsupported answers	Whether successful-looking responses actually help.
Token usage, attempts, and cost per task	Where resources are spent, including failed work.
Retrieval freshness and tool outcomes	Whether dependencies supply current facts and perform intended actions.

HTTP 200 means the transport operation succeeded under that endpoint's contract; it does not prove that a recommended film meets the user's constraints. Combine operational measurements with sampled, reviewed quality checks. An average can hide slow requests, so inspect distributions and percentiles as well.

▼ Go deeper: useful telemetry without unbounded labels

Metric labels create separate time series. A small set of version or status labels can be useful; putting every unique prompt or user ID in a metric label can create unbounded cardinality and expose personal data. Use access-controlled traces or logs for individual investigation, with sampling and retention appropriate to the task. Preserve failure counts independently of trace sampling. Distinguish a missing observation from a successful request, and avoid logging secrets in error messages or exported reports.

5. Define service objectives and error budgets

A **service-level indicator (SLI)** is a measurement of service behavior. A **service-level objective (SLO)** is the target for that indicator over a defined window. A **service-level agreement (SLA)** is an external agreement that may specify consequences. They are related, but not interchangeable. Google's [SLO workbook](#) emphasizes measurements that reflect the user experience.

For our teaching example, a **good request** returns a correct movie recommendation within the selected total deadline. The target is at least 95% good requests in the simulated 20-request window. All requests count, including errors and timeouts.

$$\text{good-request SLI} = \frac{\text{good requests}}{\text{eligible requests}}, \quad \text{allowed bad requests} = N(1 - \text{SLO target}).$$

At $N = 20$ and a target of 0.95, the **error budget** allows one bad request. Nineteen good requests meet the target exactly; sixteen do not. Four bad requests leave a budget balance of $1 - 4 = -3$: the allowance has been exceeded by three. The 95% target and tiny window are teaching choices, not production recommendations.

Define how the budget affects decisions, such as slowing feature releases while reliability is repaired. A release can also have stricter stop criteria: our lab rolls back on a known bad candidate result even when the overall SLO is still met. The budget is not permission to cause a security breach. Real services need representative volumes, defined exclusions, appropriate windows, and explicit treatment of delayed quality labels.

6. Recover without amplifying failures

A **timeout** limits waiting for an operation. An overall **deadline** limits the complete request, including earlier work and retries. A **retry** repeats a failed operation; **backoff** waits between attempts, and **jitter** varies that delay so many callers do not retry together. Bound attempts and total time, and check existing SDK retry behavior to avoid multiplying retries across layers. [AWS's retry guidance](#) explains this load amplification risk.

➤ **A retry shares the original request deadline.**



2400 ms budget: completes · 1200 ms budget: times out

More attempts consume time and resources. They do not repair a bad answer.

Figure 3. A transient failure costs 400 ms, backoff costs 200 ms, and the candidate retry needs 900 ms. It completes in 1500 ms under a 2400 ms deadline, but exceeds a 1200 ms deadline.

Retry only failures that may recover and operations that can be repeated safely. Authentication failures, invalid inputs, and policy denials usually require a different response. Follow the dependency's error semantics and any applicable retry delay. An unsuitable recommendation returned with HTTP 200 is a quality problem, not automatically a transport retry.

For a write, a timeout does not tell you whether the server already applied the change. Use a stable idempotency key and result reconciliation where supported so retrying a watchlist addition does not duplicate its effect. Keep authorization checks in place. A client timeout also does not guarantee that a provider stopped processing or billing.

▼ Go deeper: circuit breakers and graceful degradation

A circuit breaker can stop sending calls to a repeatedly failing dependency. In the closed state, calls flow; in the open state, they are rejected quickly; in a half-open state, limited probes test recovery. Configure thresholds and recovery behavior from evidence. A fallback might offer a clearly labeled catalog-only recommendation when personalized generation is unavailable. Evaluate fallback quality and permissions too; do not silently treat stale availability as current. Avoid swapping to an untested provider that changes privacy or output behavior. The lab demonstrates bounded retry and rollback, not a circuit breaker or fallback implementation.

7. Control capacity, latency, and cost together

Horizontal scaling adds service instances; **vertical scaling** gives an instance more resources. More backend instances cannot overcome an unchanged provider quota. Limit concurrency, use bounded queues, and apply **backpressure** so upstream callers slow down or receive a clear busy response when the service cannot keep up.

Technique	Benefit	What to verify
Result caching	Reuses a valid previous answer.	User permissions, region, catalog version, and freshness belong in the design.
Prompt-prefix caching	Can reduce repeated prefix processing in supported services.	It does not reuse or guarantee the final answer.
Batching	Groups suitable work for efficiency.	Waiting for a batch can increase latency; not all APIs offer the same behavior.
Streaming	Shows output as it arrives.	Faster first text does not mean the task has finished successfully.
Smaller models or shorter context	Can reduce resource use.	Rerun quality and safety evaluations for the change.

Track cost per attempted task as well as cost per successful task. Include retries, failed calls, retrieval, storage, observability, and infrastructure where relevant. Set spending controls that actually enforce the application’s policy; a notification alone does not stop traffic. Load-test realistic prompt lengths and tool paths, not only the shortest successful request.

For long jobs, return a job ID, expose status, and design cancellation, retry, and completion handling. Bound queue size and job lifetime so a temporary outage does not create an unmanageable backlog. The simulator uses fictional attempt units and sequential requests; it does not predict a provider’s price or your production capacity.

8. Operate with health checks, runbooks, and feedback

Distinguish **liveness** from **readiness**. Liveness asks whether a process needs restarting; readiness asks whether it can currently receive traffic. Startup checks allow initialization time. A brief model-provider outage should not necessarily restart every healthy application process. Kubernetes documents these [different probe purposes](#); the distinction is useful beyond Kubernetes.

An actionable alert identifies user impact, the affected service, and the responsible owner. A **runbook** gives concrete diagnosis and recovery steps: inspect the deployed version and failing dependency, reduce exposure, verify recovery, and communicate status. Page someone for urgent impact; use a lower-urgency channel for trends that can wait.

In our movie example, wrong recommendations only on the new release suggest investigating that release. Provider errors on both old and new versions suggest a shared dependency. Rollback can contain the first problem while leaving the second untouched. After recovery, record the cause and contributing conditions, add regression cases, and update the runbook. Keep data/index refreshes, credential rotation, and dependency changes in the same controlled process.

▼ Go deeper: drift and training-serving consistency

Data drift means the input distribution changes; concept drift means the relationship between inputs and the desired outcome changes. For example, a growing share of regional-language requests changes the workload, while changing viewer tastes can change what counts as a good recommendation. Drift is a signal to investigate, not an automatic command to retrain. Training-serving skew occurs when training and production features or transformations differ. Preserve feature definitions, preprocessing, and data validation across both paths. Monitor labels and quality when they become available, investigate shifts, and evaluate a proposed fix before deployment. Google's [production ML monitoring material](#) discusses these consistency and monitoring concerns.

9. Run the Python operations workbook

The four files are `operations.py`, `data.json`, `test_operations.py`, and a README. Python's standard library is sufficient. You are running a deterministic controller over authored requests; no model, network, cloud deployment, real billing, or watchlist modification occurs.

```
python operations.py
python operations.py --rollback observe
python operations.py --share 100
python operations.py --scenario provider_fault --rollback observe --retries 0
python operations.py --scenario provider_fault --rollback observe --retries 1
python operations.py --scenario provider_fault --rollback observe --deadline 1200
python -m unittest -v test_operations.py
```

Read `run_request()` to follow attempt timing and the shared deadline. `evaluate()` routes four windows of five requests, checks candidate outcomes at each window end, and applies rollback to later windows. `summarize()` counts all final results; p95 uses the nearest-rank convention from chapter 34, including failures.

In the bad-release scenario, candidate slots 1 and 3 have incorrect recommendation labels even though the response status is 200. In the provider-fault scenario, slot 1 fails once and can recover; slot 3 fails on both attempts for either version. Backoff is a fixed 200 ms. Every attempted call consumes its full fictional reserved cost, including timeouts. Immediate quality labels and fixed routing are simplifications; they cannot establish real-world reliability or statistical confidence.

10. Experiment with the release simulator

Twenty requests. A release decision.

Worked rollout: twenty movie requests, bad candidate release, 2400 ms deadline. All requests return HTTP 200, but some recommendations are wrong.

Policy	Good / 20	Candidate requests	Cost units
20% share, observe only	16/20	4	56
20% share, automatic rollback	19/20	1	59
100% share, automatic rollback	18/20	5	55

Rollback occurs after the first five-request window. Default 19/20 meets the teaching 95% SLO but stops the release. Under shared provider faults, one retry improves 12/20 to 16/20 good at 2400 ms; at 1200 ms it remains 12/20. Attempts increase from 20 to 28.

Exercise 1: find the misleading green signal. Keep the bad release and 20% candidate traffic. Compare automatic rollback with observe-only, then try 100% traffic with rollback.

▼ Reveal the reasoning

All 20 requests return HTTP 200. Automatic rollback at 20% gives 19/20 good requests and exposes one request to candidate. Observe-only gives 16/20 good with four candidate requests. At 100%, five requests reach candidate before the first check and two are bad, leaving 18/20 good. Rollback does not erase earlier results.

Exercise 2: count the price of retrying. Select provider faults, 20% traffic, observe-only, and 2400 ms. Compare zero and one retry, then lower the deadline to 1200 ms.

▼ Reveal the reasoning

One retry improves good requests from 12/20 to 16/20, while attempts increase from 20 to 28. Persistent failures remain. At 1200 ms, transient retries run out of time, leaving 12/20 good despite the extra attempts. Earlier work and backoff consume part of the deadline.

Exercise 3: separate release trouble from dependency trouble. Select provider faults, 60% traffic, one retry, 2400 ms, and automatic rollback. Inspect requests after window 1.

▼ Reveal the reasoning

A failed candidate result triggers rollback, but later baseline requests still hit the shared provider failure. Four requests remain bad across the run. Restoring old application code cannot repair an external dependency. Inspect both versions and follow the appropriate recovery runbook.

11. Check your understanding

01 Every request returns HTTP 200, but several recommendations violate user preferences. What follows?

- A. Transport success alone is insufficient; measure task quality as well.
- B. The release is reliable because the server is reachable.
- C. Retry every successful HTTP response.

Answer: A. An application can be available while giving bad answers. Define useful outcomes and measure them separately from transport status.

02 What must a retry budget include?

- A. Only the duration of the final attempt.
- B. Earlier attempts, backoff, and the remaining overall deadline.
- C. Unlimited attempts while the user is still waiting.

Answer: B. Resetting the deadline on each attempt can turn a bounded request into a long wait. Retry only appropriate errors and safe operations.

03 When should a canary be automatically expanded to everyone?

- A. After any five requests pass.
- B. Whenever it costs less than baseline.
- C. After sufficient representative evidence meets predeclared criteria; the tiny lab cannot establish that.

Answer: C. Canaries limit exposure but need enough relevant observations and quality checks. No small fixture run is a universal promotion rule.

04 The 95% teaching SLO is met at 19/20, but the candidate has one known bad result. Can the rollout stop?

- A. No, an SLO requires every release to continue until it is missed.
- B. Yes; a rollout stop rule and a window-level error budget serve different purposes.
- C. Only if all HTTP requests fail.

Answer: B. The lab stops the release on a candidate failure even while the aggregate SLO is still met. Earlier failures remain in the totals.

05 Rolling back the candidate leaves provider errors on baseline. What should you investigate?

- A. Only the candidate prompt.
- B. Whether more retries can run without limits.
- C. The shared dependency and the runbook for containment or a tested degraded mode.

Answer: C. Rollback helps when the changed release causes the fault. It cannot restore an external provider used by both versions.

12. Revision and the next connection

Follow the operating loop: **version** → **test** → **release gradually** → **observe** → **contain failures** → **improve**. Measure useful outcomes, keep deadlines and budgets explicit, and make recovery a tested capability. A working demo becomes a dependable service through repeated evidence and controlled change.

▼ Quick-reference glossary

Term	Meaning in this chapter
Release manifest	Record tying an artifact to code, configuration, data, and evaluation versions.
Canary	Limited exposure to a candidate release before wider rollout.
Rollback	Return future execution to a compatible earlier version.
Observability	Using signals to understand internal behavior and user outcomes.
Trace / span	A request path / an individual operation within it.
SLI / SLO	A service measurement / its target over a defined window.
Error budget	The amount of unreliability permitted by an SLO.
Deadline	Limit for the complete request, including attempts and waits.
Backoff / jitter	Delaying retries / varying the delay to avoid synchronization.
Circuit breaker	Temporarily limit calls to a failing dependency.
Backpressure	Slow or reject incoming work when capacity is constrained.
Runbook	Documented steps for diagnosis, containment, and recovery.
MLOps	Practices for managing the lifecycle of deployed ML systems.

Primary readings are linked alongside the concepts: OpenAI production guidance, Google SRE, OpenTelemetry, AWS reliability guidance, Kubernetes probes, and Google’s production ML material. The movie scenarios, timing assumptions, cost units, and rollout controller are original teaching examples. This chapter teaches deployment and operations; its workbook does not deploy a service.

Next: Chapter 37 - Classical, symbolic, and probabilistic AI. Broaden the picture beyond neural networks with search, planning, logic, and reasoning under uncertainty.